

DocuMind Enterprise: Sistema Inteligente de Gestión y Análisis Documental

Fase 07: Manual técnico y de administración

Juan Esteban Rojas Meléndez

Tabla de Contenidos

1. Propósito y Alcance del Manual de Administración
2. Topología y Arquitectura del Sistema Desplegado
3. Requisitos de Infraestructura y Servidor Anfitrión
4. Puesta en Marcha, Inicialización y Arranque de Servicios
5. Gestión y Configuración de Parámetros Operativos (.env)
6. Administración del Motor de Base de Datos y Vectores (pgvector)
7. Monitoreo de Procesos, Bitácoras y Gestión de Trazabilidad (error_log)
8. Mantenimiento, Optimización e Indexación Vectorial
9. Protocolos de Respaldo Lógico y Recuperación ante Desastres (Disaster Recovery)
10. Matriz de Diagnóstico y Resolución de Incidentes Técnicos (Troubleshooting)

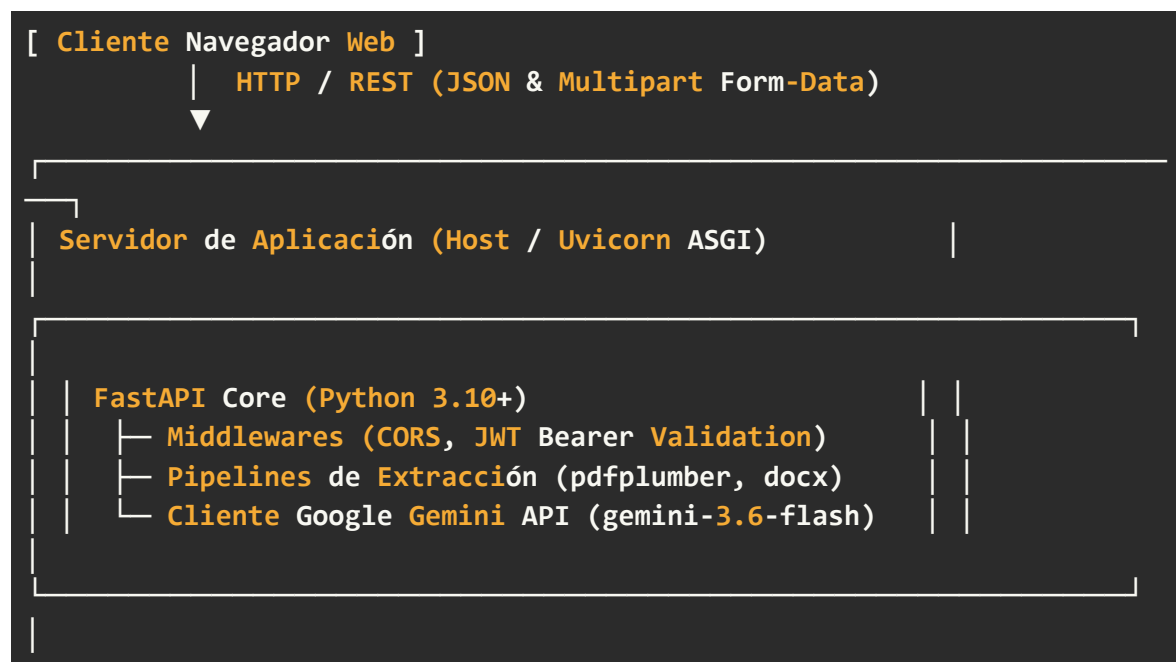
1. Propósito y Alcance del Manual de Administración

El presente documento constituye la guía técnica especializada para el Administrador del Sistema, Ingenieros de Soporte y personal de Operaciones de TI encargados de la instalación, supervisión, mantenimiento y aseguramiento de la plataforma DocuMind Enterprise.

Su cobertura abarca desde el aprovisionamiento de la máquina virtual o servidor físico anfitrión, la orquestación del contenedor PostgreSQL con soporte de aproximación vectorial (pgvector), la gestión de variables de entorno seguras, hasta los procedimientos de afinamiento (tuning) de consultas vectoriales HNSW/IVFFlat y mitigación de fallos de red en servicios externos de IA.

2. Topología y Arquitectura del Sistema Desplegado

La plataforma implementa una arquitectura desacoplada por capas con aislamiento de persistencia mediante Docker:





- **Front-End Web:** Single Page Application (SPA) construida en React y estilizada con utilidades atómicas de Tailwind CSS.
- **Back-End API:** Microservicio en FastAPI servido por el servidor asíncrono Uvicorn en el puerto 8000.
- **Base de Datos y Almacén Vectorial:** PostgreSQL 16 encapsulado en la imagen pgvector/pgvector:pg16, mapeado en el puerto TCP 5432.
- **Sistema de Archivos Físico:** Directorio backend/storage/ con políticas de lectura/escritura exclusivas para el usuario del proceso backend.

- **Capa Externa de IA:** Google AI Studio conectado vía HTTPS con TLS 1.3.

3. Requisitos de Infraestructura y Servidor Anfitrión

3.1 Especificaciones de Hardware del Servidor

- **CPU:** Mínimo 2 núcleos físicos o 2 vCPU arquitectura x86_64 con soporte de instrucciones AVX2 (recomendado para operaciones vectoriales aceleradas en memoria).
- **Memoria RAM:** Mínimo 4 GB dedicados (8 GB recomendados para soportar buffers compartidos de PostgreSQL, Uvicorn e indexación de fragmentos).
- **Almacenamiento Secundario:** 15 GB de espacio libre en discos de estado sólido (SSD NVMe preferentemente) para el volumen de Docker y almacenamiento de archivos binarios originales.
- **Red:** Interfaz Ethernet Gigabit con salida a Internet habilitada sobre los puertos TCP 80 y 443 (para llamadas al endpoint `generativelanguage.googleapis.com`).

3.2 Plataforma de Software

- **Sistema Operativo Homologado:** Ubuntu Server 22.04 LTS, Debian 12, o Windows Server 2022 / Windows 11 con WSL2.
- **Docker Engine:** Versión 24.0.0 o superior junto con Docker Compose v2.20+.
- **Intérprete Python:** Versión 3.10.x o 3.11.x compilada para 64 bits.
- **Node.js Runtime:** Versión 20 LTS o superior.

4. Puesta en Marcha, Inicialización y Arranque de Servicios

Paso 1: Configuración de la Red y Contenedor de Base de Datos

1. Inicie la máquina virtual o terminal anfitriona y clone el repositorio oficial:
- 2.

```
Bash
git clone https://github.com/usuario/DocuMind-Enterprise.git
cd DocuMind-Enterprise
```

3. Despliegue el contenedor en segundo plano mediante Docker Compose:

4.

```
Bash
docker compose up -d
```

5. Compruebe que el servicio documind_db esté levantado y acepte conexiones:

6.

```
Bash
docker ps --filter "name=documind_db"
```

7. Inicialice obligatoriamente la extensión vectorial pgvector en la base de datos empresarial:

8.

```
Bash
docker exec -i documind_db psql -U postgres -d documind -c "CREATE
EXTENSION IF NOT EXISTS vector;"
```

Paso 2: Aprovisionamiento y Ejecución del Backend (FastAPI)

1. Navegue al directorio backend/:

2.

```
Bash
cd backend
```

3. Instancie el entorno virtual de ejecución y actívelo:

4.

```
Bash
python -m venv venv
# En Linux / macOS:
source venv/bin/activate
# En Windows PowerShell:
.\venv\Scripts\activate
```

5. Instale el árbol de dependencias certificado en requirements.txt:

6.

```
Bash
pip install --upgrade pip
pip install -r requirements.txt
```

7. Ejecute el script DDL de creación de tablas relacionales e índices vectoriales:

8.

```
Bash
python create_tables.py
```

9. Inicie el servidor de aplicaciones ASGI:

10.

```
Bash
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 2
```

Paso 3: Puesta en Marcha del Frontend

1. En una consola secundaria, sitúese en la carpeta frontend/:

2.

```
Bash
cd ../frontend
```

3. Instale los módulos de Node.js:

4.

```
Bash  
npm install
```

5. Ejecute el servidor de desarrollo Vite o compile para producción:

6.

```
Bash  
# Modo desarrollo:  
npm run dev -- --host 0.0.0.0 --port 5173  
# Modo compilación estática de producción:  
npm run build
```

5. Gestión y Configuración de Parámetros Operativos (.env)

La parametrización del sistema sigue la metodología Twelve-Factor App (RNF-01).

El archivo backend/.env debe mantenerse con permisos de lectura restrictivos (chmod 600 .env en entornos Unix):

```
Fragmento de código  
# =====  
# CADENA DE CONEXIÓN A POSTGRESQL (DOCKER)  
# =====  
DATABASE_URL=postgresql://postgres:admin@localhost:5432/documind  
  
# =====  
# CREDENCIALES Y MODELOS DE INTELIGENCIA ARTIFICIAL  
# =====  
GEMINI_API_KEY=AIzaSyA_TU_LLAVE_PRIVADA_AQUI  
GEMINI_TEXT_MODEL=gemini-3.6-flash  
GEMINI_EMBEDDING_MODEL=models/gemini-embedding-2  
  
# =====  
# CRIPTOGRAFÍA Y CONTROL DE ACCESO (JWT)
```

```
# =====
JWT_SECRET_KEY=documind_enterprise_secure_token_2026_uts
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=120

# =====
# REGLAS DE NEGOCIO Y POLÍTICAS DE ALMACENAMIENTO
# =====
STORAGE_DIR=storage/
MAX_FILE_SIZE_BYTES=10485760
ALLOWED_EXTENSIONS=pdf,docx,txt
```

6. Administración del Motor de Base de Datos y Vectores (pgvector)

6.1 Estructura del Esquema de Base de Datos

El esquema combina tablas relacionales ACID con soporte vectorial multidimensional:

- Tabla documents:
 - id (Integer, Primary Key)
 - filename (VARCHAR(255), Not Null)
 - file_path (VARCHAR(500), Not Null)
 - mime_type (VARCHAR(100), Not Null)
 - file_size_bytes (BigInteger, Not Null)
 - status (VARCHAR(50), Default 'Pendiente')
 - category (VARCHAR(100), Default 'Sin Clasificar')
 - summary (TEXT)
 - extracted_data (JSONB)
 - error_log (TEXT)
 - created_at (TIMESTAMP WITH TIME ZONE)
- Tabla document_chunks:
 - id (Integer, Primary Key)

- document_id (Integer, Foreign Key a documents.id ON DELETE CASCADE)
- chunk_index (Integer, Not Null)
- chunk_text (TEXT, Not Null)
- embedding (Vector(768), Not Null)

6.2 Comandos de Verificación Administrativa

Para ingresar a la consola SQL del motor contenerizado:

```
Bash
docker exec -it documind_db psql -U postgres -d documind
```

- Comprobar la existencia de la extensión vector:

-

```
SQL
\dx vector
```

- Verificar el conteo y distribución de documentos procesados:

-

```
SQL
SELECT category, status, COUNT(*)
FROM documents
GROUP BY category, status;
```

- Comprobar la dimensión exacta de los vectores almacenados:

-

```
SQL
SELECT chunk_index, vector_dims(embedding)
FROM document_chunks
LIMIT 5;
```

- (La consulta debe devolver exactamente 768 dimensiones).

7. Monitoreo de Procesos, Bitácoras y Gestión de Trazabilidad (error_log)

El ciclo de vida del procesamiento documental transiciona de forma obligatoria por los estados: Pendiente → Procesando → Procesado / Error (RN-05, RF-18).

Pendiente → Procesando → Procesado / Error (RN-05, RF-18).

7.1 Auditoría de Fallos Técnicos

Cuando un documento falla durante la extracción de texto plano o al invocar la API de Gemini, el backend nunca colapsa; conmuta el estado del archivo a Error y almacena la traza técnica de depuración en la columna error_log.

Para auditar y listar las causas de fallo en el sistema:

```
SQL
SELECT id, filename, mime_type, error_log
FROM documents
WHERE status = 'Error';
```

7.2 Inspección de Logs en Vivo

- Logs del Servidor FastAPI: Permite visualizar en tiempo real las peticiones entrantes, códigos HTTP de retorno y trazas de Python:

-

```
Bash
tail -f -n 100 backend/uvicorn.log
```

- Logs del Motor de Base de Datos (Docker): Permite auditar bloqueos, transacciones abortadas o caídas de conexión:

-

```
Bash
docker logs -f --tail 50 documind_db
```

8. Mantenimiento, Optimización y Reindexación Vectorial

A medida que el acervo documental supera los 1.000 fragmentos (chunks), la búsqueda semántica secuencial en memoria puede degradar los tiempos de respuesta exigidos en el RNF-03 (< 5 s).

8.1 Creación y Mantenimiento del Índice HNSW (RNF-05)

Para optimizar las consultas por distancia de coseno ($\cos$) en pgvector, el administrador debe crear un índice de grafos HNSW (Hierarchical Navigable Small World):

```
SQL
CREATE INDEX IF NOT EXISTS idx_document_chunks_embedding_hnsw
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

8.2 Rutina Periódica de Mantenimiento (Ejecución Mensual)

Se debe ejecutar un proceso de limpieza de espacio muerto y actualización de estadísticas del planificador de PostgreSQL:

```
SQL
VACUUM ANALYZE documents;
VACUUM ANALYZE document_chunks;
REINDEX TABLE document_chunks;
```

9. Protocolos de Respaldo Lógico y Recuperación ante Desastres (Disaster Recovery)

9.1 Política de Copias de Seguridad (Backups)

Se establece un esquema de respaldo diario compuesto por dos artefactos inseparables:

1. Respaldo Lógico de Datos y Vectores (PostgreSQL):

- 2.

```
Bash
docker exec -t documind_db pg_dump -U postgres -d documind
--format=custom -f /tmp/db.dump
docker cp documind_db:/tmp/db.dump ./backups/db_documind_$(date
+%Y%m%d_%H%M%S).dump
docker exec documind_db rm /tmp/db.dump
```

3. Respaldo de Archivos Fuente Binarios:

4.

```
Bash
tar -czf ./backups/storage_binaries_$(date +%Y%m%d_%H%M%S).tar.gz -C
backend/ storage/
```

9.2 Plan de Restauración ante Caída Crítica (Disaster Recovery)

En caso de corrupción de hardware o pérdida total del servidor host:

1. Instale un nuevo ambiente Docker en el servidor de contingencia.
2. Inicie el contenedor limpio mediante `docker compose up -d`.
3. Restaure el esquema y los vectores desde el archivo de backup:

4.

```
Bash
docker cp ./backups/db_documind_YYYYMMDD_HHMMSS.dump
documind_db:/tmp/db.dump
docker exec -i documind_db pg_restore -U postgres -d documind --clean
--if-exists /tmp/db.dump
```

5. Descomprima los binarios en la ruta de trabajo del backend:

6.

```
Bash
tar -xzf ./backups/storage_binaries_YYYYMMDD_HHMMSS.tar.gz -C backend/
```

7. Inicie el servicio FastAPI y compruebe la integridad en `http://localhost:8000/docs`.

Tabla 1

Matriz de Diagnóstico y Resolución de Incidentes Técnicos (Troubleshooting)

10. Matriz de Diagnóstico y Resolución de Incidentes Técnicos (Troubleshooting)

Incidente / Alarma	Causa Raíz Técnica	Acciones de Mitigación y Recuperación
HTTP 500 al subir documentos (Failed to fetch)	El backend se detuvo tras un fallo fatal o el script no tiene permisos en backend/storage/.	1. Verificar permisos del directorio: <code>chmod 755 backend/storage</code>. 2. Revisar si el entorno virtual está activo. 3. Reiniciar Uvicorn validando que no existan puertos en conflicto (Isot -i :8000).
HTTP 429 Too Many Requests en la carga de lotes	Se superó el límite de peticiones por minuto (RPM) de la capa gratuita de Google AI Studio.	1. Implementar throttler o pausas de 4 segundos entre documentos. 2. Procesar lotes secuencialmente en lugar de envíos paralelos masivos. 3. Asociar facturación en Google Cloud Console para habilitar cuota Pay-as-you-go.
Error: vector dimensions do not match: expected 768, got 3072	El modelo gemini-embedding-2 generó vectores en su dimensión nativa en lugar del límite compatible.	1. Modificar la llamada de embedding en el backend. 2. Añadir explícitamente el parámetro <code>output_dimensionality=768</code> en la petición a la API. 3. Reiniciar el backend.
El chat RAG responde que no tiene datos sobre un archivo subido	El archivo falló durante la vectorización o no se generaron chunks en document_chunks.	1. Consultar la tabla documents para verificar que status = 'Procesado'. 2. Verificar en la base de datos: <code>SELECT COUNT(*) FROM document_chunks WHERE document_id = X;</code> 3. Si el conteo es 0, volver a cargar el archivo.

Contenedor Docker en estado Exited (1) o reiniciándose	Conflicto en el puerto local 5432 (otro servicio de PostgreSQL local corriendo) o volumen corrupto.	1. Detener instancias nativas de PostgreSQL en el host (sudo service postgresql stop o detener el servicio en services.msc). 2. Reejecutar docker compose up -d.
--	---	--
